

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

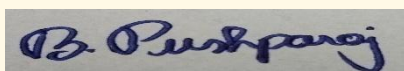
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA1008	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	B.Pushparaj	Reg. No.:	192421136
Date:	19.8.2026	Duration:	60 minutes

Code of Conduct

I B. Pushparaj (192421136) that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____



Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a Code Review and Repository Evaluation of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

2. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

2. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

2. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

2. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
- Suggest improvements to enhance usability and maintainability.

2. Code Review Findings and Recommendations

- Summarize the overall code review findings.
- Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
---------------------	---------------------	----------------	------------------------	----------------------------

Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

	are complete, clear, and well organized.			
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Project Topic: Accessible Learning Platform

1. Problem Overview

Problem Statement

Students with visual impairments face difficulties using conventional online learning platforms because many systems do not provide proper accessibility features. The proposed **Accessible Learning Platform** provides an inclusive learning environment with screen-reader support, voice navigation, audio learning resources, accessible assessments, and progress tracking.

Project Objectives

- Provide an accessible online learning environment.
- Support screen readers and keyboard navigation.
- Provide audio-based learning materials.
- Allow students to access courses and assessments easily.
- Help teachers manage courses and learning content.
- Track student progress.
- Improve usability for students with different accessibility needs.

Key Functionalities

Module	Function
User Management	Student/teacher registration and login
Course Management	Create, view and manage courses
Accessible Learning	Text, audio and screen-reader-friendly content
Assessment	Accessible online tests
Progress Tracking	Monitor completed courses and scores
Teacher Module	Manage learning materials
Admin Module	Manage users and system information

2. Repository Organization

The proposed repository structure is:

Accessible-Learning-Platform/

|

```
|— app.py
|— requirements.txt
|— Dockerfile
|— docker-compose.yml
|— README.md
|
|— database/
|   |— database.sql
|
|— templates/
|   |— index.html
|   |— login.html
|   |— register.html
|   |— dashboard.html
|   |— courses.html
|   |— assessment.html
|   |— progress.html
|
|— static/
|   |— css/
|       |— style.css
|
|   |— js/
|       |— script.js
```

Purpose of Files

File/Folder	Purpose
app.py	Main Flask application
requirements.txt	Python dependencies
Dockerfile	Creates application container
docker-compose.yml	Runs application and database services

README.md	Project documentation
database/	Database-related files
templates/	HTML pages
static/css/	Website styling
static/js/	Client-side functionality

Repository Evaluation

The structure is modular because HTML, CSS, JavaScript, backend code, and database files are separated. This makes the project easier to understand, maintain, debug, and modify.

3. Code Quality Review

3.1 Readability

The code should use:

- Meaningful variable names.
- Proper indentation.
- Small and understandable functions.
- Comments for important logic.
- Consistent naming conventions.

Example

```
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]

        # Validate user credentials
        user = authenticate_user(username, password)

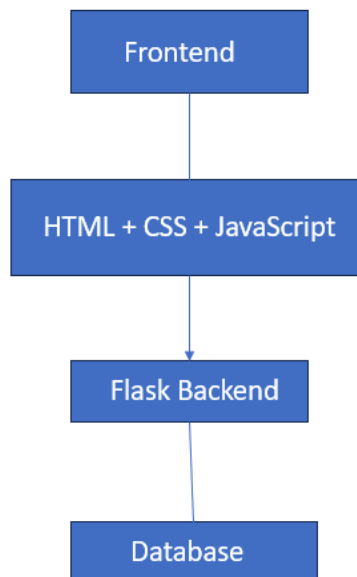
        if user:
            return redirect("/dashboard")

    return render_template("login.html")
```

This code is easy to understand because the function name and variables clearly describe their purpose.

3.2 Modularity

The project separates different responsibilities:



This makes it easier to modify one component without affecting the complete system.

3.3 Coding Standards

Recommended practices:

- Use lowercase names for Python functions.
- Use descriptive variable names.
- Avoid unnecessary duplicate code.
- Keep functions short.
- Use comments where required.
- Keep HTML, CSS and JavaScript separate.

3.4 Code Quality Strengths

1. Simple project structure.
2. Separate frontend and backend.

3. Modular HTML templates.
4. Accessibility-focused interface.
5. Docker support.
6. Git-based version control.
7. Easy future expansion.

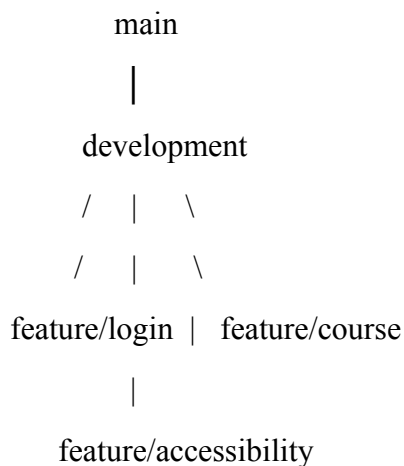
Areas for Improvement

1. Add stronger input validation.
2. Improve password security.
3. Add automated unit testing.
4. Add error logging.
5. Separate Flask routes into multiple modules as the project grows.
6. Add API documentation.

4. Version Control Evaluation

Git is used to maintain different versions of the Accessible Learning Platform.

Branching Strategy



Main Branch

Contains stable versions of the application.

Development Branch

Contains combined development work before release.

Feature Branches

feature/login

feature/accessibility

feature/course-management

Each feature can be developed independently.

Git Commands Used

Initialize Repository

git init

Check Repository

git status

Add Files

git add .

Commit

git commit -m "Initial Accessible Learning Platform setup"

Create Development Branch

git checkout -b development

Create Feature Branch

git checkout -b feature/login

Merge Feature

git checkout development

git merge feature/login

View History

git log --oneline --graph --all

5. Commit History Evaluation

Good commit messages should clearly describe the change.

Example Commit History

Initial Accessible Learning Platform setup

Add student and teacher login

Add accessible navigation

Add audio learning support

Add course management module

Add accessible assessment module

Add Docker configuration

Update project documentation

Evaluation

The commit messages are:

- Short.
- Meaningful.
- Feature-specific.
- Easy to understand.
- Useful for tracking project changes.

This helps developers identify when and why a feature was added or modified.

6. Code Testing and Validation

The application should be tested using different functional test cases.

Test Cases

Test ID	Test Case	Expected Result	Status
TC01	Open application	Home page displayed	Pass
TC02	Student registration	Account created	Pass
TC03	Valid login	Dashboard displayed	Pass
TC04	Invalid login	Error displayed	Pass
TC05	Open course	Course content displayed	Pass
TC06	Play audio lesson	Audio starts	Pass
TC07	Take assessment	Questions displayed	Pass
TC08	Submit assessment	Score displayed	Pass
TC09	View progress	Progress displayed	Pass
TC10	Logout	User logged out	Pass

In the final report, replace the example **Pass** values with the results from your actual execution.

7. Accessibility Testing

The main special feature of this project is accessibility.

Test 1 – Keyboard Navigation

Check whether users can navigate through the platform using:

Tab → Enter → Arrow Keys

Test 2 – Screen Reader

Verify that important elements such as:

Login

Course

Audio

Assessment

Submit

Logout

are properly announced.

Test 3 – Audio Learning

Verify that audio learning material can be played successfully.

Test 4 – Accessible Assessment

Check whether assessment questions and options can be accessed without depending only on a mouse.

8. Repository Documentation

The repository should contain a README.md.

README Contents

1. Project Title
2. Problem Statement
3. Objectives
4. Features
5. Technologies Used
6. Project Structure
7. Installation
8. Running the Application

9. Docker Instructions
10. Testing
11. Screenshots
12. Future Enhancements

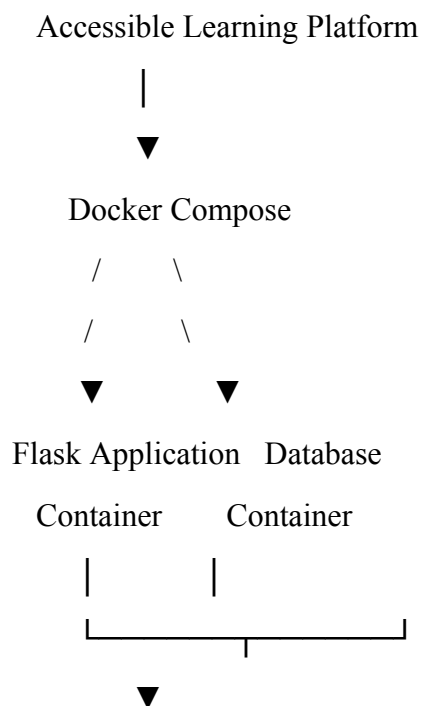
Technologies

Technology	Purpose
Python	Backend
Flask	Web framework
HTML	User interface
CSS	Styling
JavaScript	Interactive functions
MySQL/SQLite	Database
Git	Version control
Docker	Containerization

9. Docker Evaluation

Docker makes the application easier to run on different computers.

Docker Architecture



Web Browser

Dockerfile

A basic Dockerfile can be:

FROM python:3.12

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]

Docker Compose

Example:

services:

web:

build: .

ports:

- "5000:5000"

depends_on:

- db

db:

image: mysql:8.0

environment:

MYSQL_ROOT_PASSWORD: root

MYSQL_DATABASE: accessible_learning

10. Code Review Findings

Strengths

- Clear project purpose.
- Simple repository organization.
- Modular frontend.
- Flask backend is easy to maintain.
- Git supports feature-based development.
- Docker improves portability.
- Accessibility is considered as a core requirement.

Weaknesses

- Automated testing can be improved.
- Security needs further enhancement.
- Large applications may require more backend modules.
- Database error handling can be improved.
- Accessibility testing should be more comprehensive.

11. Recommendations

Code Quality

- Follow PEP 8 coding practices.
- Use reusable functions.
- Add error handling.
- Add automated tests.

Git

- Use meaningful commit messages.
- Protect the main branch.
- Use pull requests for feature merging.
- Review code before merging.

Documentation

- Maintain an updated README.
- Add installation instructions.
- Document database configuration.
- Document Docker commands.

Security

- Hash passwords.
- Validate user input.
- Use secure sessions.
- Store sensitive configuration in environment variables.

Accessibility

- Improve screen-reader compatibility.
- Add keyboard shortcuts.
- Provide captions/transcripts.
- Improve color contrast.
- Add alternative text for images.

12. Future Enhancements

1. Mobile application support.
2. More audio learning resources.
3. Voice-based navigation.
4. Multilingual learning content.
5. Advanced accessibility testing.
6. Teacher analytics dashboard.
7. Cloud deployment.
8. Automated CI/CD using GitHub Actions.
9. Better notification system.
10. Offline learning support.

13. Overall Code Review Conclusion

The **Accessible Learning Platform** provides a practical solution for making online education more inclusive. The project uses a structured repository, Git-based version control, modular code organization, testing, and Docker containerization.

The main strength of the project is its focus on **accessibility**, particularly screen-reader-friendly content, keyboard navigation, audio learning, and accessible assessments. With improvements in automated testing, security, documentation, and deployment, the system can become more reliable and maintainable.